FEM vs. FDM Poisson Comparison

Technical Specification: Structure and Mathematical Correctness

`src/compare` module
Numerical-PDE

October 26, 2026

Abstract

This document specifies the `src/compare` module: a self-contained comparison of the finite-difference method (FDM) and the finite-element method (FEM) applied to the Poisson equation on four preset two-dimensional domains. It covers the module's file structure, the continuous problem being solved, the derivation of both discretizations from first principles, the linear-algebra machinery (CSR storage and Conjugate Gradient) used to solve the resulting systems, and the argument for why the two independently-derived discretizations are expected to agree, which is what makes the comparison a meaningful correctness check rather than just two numbers placed side by side. It also covers the standalone interactive desktop window (`src/compare/gui`) that lets a user change the domain, f, the mesh resolution, and the method, then re-solve and re-render on demand – including the software axonometric projection used to draw the solution as a shaded 3D surface without an OpenGL dependency.

Contents

1 Scope and goals **2**

2 Module structure **2**

3 The continuous problem **3**

4 Domains and the shared grid **4**
4.1 Domain predicates 4
4.2 Grid, classification, and degrees of freedom 4

5 Finite differences: the five-point Laplacian **5**
5.1 Derivation 5
5.2 Assembly (`assembleFDM`) 5

6 Finite elements: linear (P1) Galerkin on a triangulated grid **5**
6.1 Weak form 5
6.2 Triangulation and basis 6
6.3 Element stiffness: closed form 6
6.4 Load vector: quadrature 6
6.5 Isolated-node guard 7

7 Boundary conditions: symmetric Dirichlet elimination **7**

8 Linear algebra **7**
8.1 CSR storage 7
8.2 Conjugate Gradient 8

9 Why the comparison is a correctness check **8**

10 The interactive desktop window **9**
10.1 Controls and the solve-on-demand model 9
10.2 Rendering: two views of the same vector 9

11 Known approximations and limitations **10**

1 Scope and goals

The module solves the scalar Poisson problem

$$-\Delta u = f \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega, \tag{1}$$

on a chosen domain $\Omega \subset [0,1]^2$, for a chosen source f, using two different numerical methods built independently on the same grid:

- a **finite-difference** discretization: the standard five-point stencil for the Laplacian on a uniform grid;
- a **finite-element** discretization: piecewise-linear (P1) Galerkin elements on a triangulation of the same grid.

Both discretizations produce a sparse linear system $A\mathbf{u} = \mathbf{b}$, stored in Compressed Sparse Row (CSR) format, and both are solved with the same hand-written Jacobi-preconditioned Conjugate Gradient (CG) iterative solver. Section 2 maps this description onto the source tree; Sections 3–8 derive the mathematics; Section 9 states precisely what "the two methods agree" means and why that is evidence of correctness rather than coincidence.

This module is deliberately narrower than the project's long-term roadmap (`docs/ROADMAP.md`), which targets a general linear-PDE framework: arbitrary order, arbitrary coefficients, time-dependence, Eigen-backed assembly, CSG geometry. Here the equation is fixed to Poisson, the geometry is four fixed presets, the linear algebra is hand-rolled CSR and CG rather than a library, and the domains are independent of the existing (and, per the roadmap's own bug inventory, unfinished) `Domain2d`/`mesh2d` code. The simplifications are intentional: a from-scratch, readable implementation of both classical discretizations for learning purposes, not a replacement for the general framework.

2 Module structure

All files below live under `src/compare/`. Table 1 gives each file's role and which section of this document derives the mathematics it implements.

Data flow. `main.cpp` builds one `Grid2d` from the chosen domain and resolution N, then calls `assembleFDM` and `assembleFEM` on that *same* grid with the *same* source expression, solves both resulting systems with `conjugateGradient`, and writes both solution fields – indexed by the same

File	Role	Derived in
`Domains.hpp`	Point-in-domain predicates for the four presets (square, L-shape, plus, circle).	§4.1
`SourceFunctions.hpp`	A library of right-hand sides $f(x, y)$, written as `pde::Expr` (the project's symbolic expression type), so f is chosen the same way any PDE coefficient is chosen elsewhere in the project.	§3
`Grid2d.hpp`	The shared $(N{+}1) \times (N{+}1)$ structured grid: classification of nodes as active/boundary and their degree-of-freedom (DOF) numbering.	§4.2
`Assemble.hpp`	`assembleFDM`, `assembleFEM`, and the shared Dirichlet-elimination pass `applyDirichletZero`.	§5, §6, §7
`CSRMatrix.hpp`	Compressed Sparse Row matrix built from triplets, with `matvec`.	§8.1
`ConjugateGradient.hpp`	Jacobi-preconditioned CG solver used for both systems.	§8.2
`main.cpp`	CLI/interactive driver: builds both systems, times and solves both, writes CSV output.	–
`plot_compare.py`	Renders the 2D heatmap comparison and the 3D surface comparison from the CSV output.	–
`gui/CompareWindow`	The standalone interactive window: domain / f / method / mesh-N / view controls, and the solve-on-demand logic calling the same assembly and CG functions as `main.cpp`.	§10
`gui/HeatmapView`	The live-redrawn viewer widget: a flat heatmap mode and a software 3D surface mode (no OpenGL dependency).	§10

Table 1: Module files and where their mathematics is specified.

grid coordinates – into one CSV file. `plot_compare.py` reads that file and renders the two fields, their pointwise difference, and the CG convergence history.

3 The continuous problem

Equation (1) is the model elliptic boundary value problem. With homogeneous Dirichlet data it is well-posed: for $f \in L^2(\Omega)$ there is a unique $u \in H_0^1(\Omega)$ satisfying the weak form (introduced formally in Section 6), and the Laplacian with this boundary condition is a symmetric, coercive, hence positive-definite operator on $H_0^1(\Omega)$. That positive-definiteness is inherited by *both* discrete operators built below – it is the reason Conjugate Gradient, which requires a symmetric positive-definite (SPD) matrix, applies to both without modification.

`SourceFunctions.hpp` fixes f to one of a small named library, each a `pde::Expr`:

Name	$f(x, y)$
`constant`	1
`sine`	$\sin(\pi x)\sin(\pi y)$
`bump`	$\exp\big(-\big[(x-\frac{1}{2})^2+(y-\frac{1}{2})^2\big]/0.02\big)$
`linear`	$x+y$
`checker`	$\sin(2\pi x)\sin(3\pi y)$

Because f is an `Expr` rather than a hard-coded C++ function, `evalF` evaluates it at arbitrary points through the project's existing symbolic evaluator (`pde::eval`, `core/Expr.hpp`), which is exactly the mechanism the rest of the project uses for PDE coefficients, sources, and boundary data. Adding a new f is one line in the library table, not a new code path.

4 Domains and the shared grid

4.1 Domain predicates

Each domain is a boolean predicate $\chi_\Omega : [0,1]^2 \to \{\text{true}, \text{false}\}$, implemented directly (`Domains.hpp`) rather than built from the project's general CSG machinery:

- **square**: $\Omega = [0,1]^2$.
- **lshape**: $\Omega = [0,1]^2 \setminus (0.5,1] \times (0.5,1]$ – the unit square with its top-right quadrant removed, producing a re-entrant corner at $(0.5, 0.5)$.
- **plus**: $\Omega = \big([0.2,0.8] \times [0.4,0.6]\big) \cup \big([0.4,0.6] \times [0.2,0.8]\big)$, the union of a horizontal and a vertical bar.
- **circle**: $\Omega = \{(x,y) : (x-0.5)^2 + (y-0.5)^2 \le 0.45^2\}$.

4.2 Grid, classification, and degrees of freedom

`Grid2d` lays a uniform $(N{+}1) \times (N{+}1)$ node grid over $[0,1]^2$, $x_i = ih$, $y_j = jh$, $h = 1/N$, and classifies every node (i,j) as:

- *inactive* (exterior) if $\chi_\Omega(x_i, y_j)$ is false – excluded from the system entirely;
- *active*, and further *boundary* if it is active but lies on the edge of the bounding grid *or* has a 4-neighbor that is inactive:

$$\text{boundary}(i,j) \iff \text{active}(i,j) \ \wedge \ \Big(i \in \{0, N\} \ \vee \ j \in \{0, N\} \\ \vee \ \exists \text{ a 4-neighbor } (i',j') \text{ with } \neg\,\text{active}(i',j') \Big);$$

- *interior* otherwise (active and not boundary) – by construction, an interior node's four grid neighbors are all active, a fact both assemblers rely on to avoid any out-of-domain stencil access.

Every active node, boundary or interior, is given a DOF index (boundary nodes are *not* eliminated from the numbering; see Section 7 for why they are kept and how the boundary condition is enforced on their rows instead). **This single `Grid2d` instance is reused for both the FDM and the FEM assembly** – both discretizations therefore produce vectors indexed by the identical set of DOFs, over the identical physical nodes. This is what lets Section 9 compare the two solution vectors node-for-node with no interpolation step, and it is the central design decision that makes the comparison meaningful.

This classification is a *staircase* ("immersed") approximation of Ω: the discrete boundary follows grid lines, not the true geometric boundary. For the square, plus, and L-shape – whose boundaries

are themselves axis-aligned – this is exact. For the circle it is an $O(h)$ approximation of the boundary location; see Section 11.

5 Finite differences: the five-point Laplacian

5.1 Derivation

For a smooth function u, the centered second-difference approximates the second derivative by Taylor expansion:

$$u(x+h) = u(x) + h\,u'(x) + \tfrac{h^2}{2}u''(x) + \tfrac{h^3}{6}u'''(x) + \tfrac{h^4}{24}u^{(4)}(x) + O(h^5), \tag{2}$$

$$u(x-h) = u(x) - h\,u'(x) + \tfrac{h^2}{2}u''(x) - \tfrac{h^3}{6}u'''(x) + \tfrac{h^4}{24}u^{(4)}(x) + O(h^5). \tag{3}$$

Adding these cancels the odd-order terms:

$$\begin{aligned} u(x+h) + u(x-h) - 2u(x) &= h^2 u''(x) + \tfrac{h^4}{12}u^{(4)}(x) + O(h^6) \\ \Longrightarrow \quad u''(x) &= \frac{u(x+h) - 2u(x) + u(x-h)}{h^2} + O(h^2). \end{aligned} \tag{4}$$

Applying (4) along each axis and summing gives the standard five-point Laplacian stencil at node (i, j) with step h in both directions:

$$(\Delta_h u)_{i,j} = \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2} = \Delta u(x_i, y_j) + O(h^2). \tag{5}$$

The scheme is therefore **second-order accurate** on a uniform grid at points whose full stencil lies in the domain of smoothness of u – i.e. at every *interior* node as defined in Section 4.2, since those nodes' four neighbors are guaranteed active.

5.2 Assembly (`assembleFDM`)

Equation (1) discretized by (5) at each interior node (i, j) with DOF d gives one row of $A\mathbf{u} = \mathbf{b}$:

$$\frac{4}{h^2}\,u_d \;-\; \frac{1}{h^2}\sum_{(i',j')\in\mathcal{N}(i,j)} u_{d'} \;=\; f(x_i, y_j), \tag{6}$$

where $\mathcal{N}(i, j)$ is the set of four grid neighbors and d' their DOF indices. The implementation emits exactly these five triplets per interior row; boundary rows are left empty at this stage and are filled in afterward by the shared elimination pass of Section 7. The resulting matrix, restricted to interior rows/columns, is the familiar block having 4 on the diagonal and -1 at each existing neighbor coupling – symmetric by construction, since the coupling from (i, j) to a neighbor and from that neighbor back to (i, j) carry the same weight $-1/h^2$.

6 Finite elements: linear (P1) Galerkin on a triangulated grid

6.1 Weak form

Multiply (1) by a test function $v \in H_0^1(\Omega)$ and integrate by parts (Green's identity), using $v = 0$ on $\partial\Omega$ to kill the boundary term:

$$\int_\Omega \nabla u \cdot \nabla v \, dA = \int_\Omega f v \, dA \qquad \forall\, v \in H_0^1(\Omega). \tag{7}$$

This is the problem's *weak form*: it only requires u to have one derivative in L^2, rather than two, which is exactly what a piecewise-*linear* approximation can supply (its second derivatives are concentrated as singular distributions on element edges and would make no sense in (1) directly). The Galerkin method picks a finite-dimensional subspace $V_h \subset H_0^1(\Omega)$ and solves (7) with u, v restricted to V_h.

6.2 Triangulation and basis

`assembleFEM` triangulates the grid of Section 4.2 directly: a cell with all four corners active is split along its BL–TR diagonal into two triangles; a cell with exactly three active corners contributes the single triangle those three corners form (so the mesh footprint tracks the staircase boundary a little more tightly than skipping every partial cell would); a cell with fewer than three active corners contributes nothing. V_h is the space of functions continuous on Ω and linear on each triangle – equivalently, the span of the *hat functions* $\{\lambda_k\}$, one per active DOF k, where λ_k is 1 at node k, 0 at every other node, and linear in between.

6.3 Element stiffness: closed form

On a single triangle with vertices p_0, p_1, p_2 (counter-clockwise) and area $A = \frac{1}{2}\big|(p_1 - p_0) \times (p_2 - p_0)\big|$, the barycentric coordinate λ_i (the hat function restricted to this triangle) is affine, so $\nabla\lambda_i$ is the constant vector

$$\nabla\lambda_i = \frac{1}{2A}\,(b_i, c_i), \qquad b_i = y_{i+1} - y_{i-1}, \quad c_i = x_{i-1} - x_{i+1} \quad \text{(indices mod 3)}. \tag{8}$$

(This is the standard P1 result: b_i, c_i are, up to sign, the coordinates of the edge opposite vertex i, rotated 90° – exactly the vector that is 0 along that opposite edge and scales linearly to 1 at vertex i.) Since each gradient is constant on the triangle, the local stiffness integral is just the gradient dot product times the area:

$$K_{ij}^{(e)} = \int_{\triangle} \nabla\lambda_i \cdot \nabla\lambda_j \, dA = \frac{b_i b_j + c_i c_j}{4A}. \tag{9}$$

This is precisely the formula implemented in `detail::addTriangle`. Each of the three local degrees of freedom contributes a 3×3 block, scattered by global DOF index into the triplet list – the same accumulate-by-summing-duplicate-triplets mechanism described in Section 8.1 handles elements sharing an edge (and hence a DOF) automatically.

6.4 Load vector: quadrature

The right-hand side of (7) restricted to one triangle is $\int_{\triangle} f\lambda_i \, dA$. Using the exact identity $\int_{\triangle} \lambda_i \, dA = A/3$ (a linear hat function averages to 1/3 of its peak value over the triangle it is one vertex of) together with a one-point (centroid) quadrature approximation of f:

$$\int_{\triangle} f\lambda_i \, dA \;\approx\; f(x_c, y_c) \cdot \frac{A}{3}, \qquad (x_c, y_c) = \tfrac{1}{3}(p_0 + p_1 + p_2). \tag{10}$$

This is exact when f is constant on the triangle and first-order accurate in general (the error is $O(h^2 A)$ per triangle from the quadratic remainder of f around the centroid, consistent with the overall discretization error of linear elements) – a standard, simple choice appropriate for the smooth source functions in `SourceFunctions.hpp`.

6.5 Isolated-node guard

A node whose only active neighbors fail to form a 3- or 4-corner cell (e.g. a single corner touching the rest of the domain only diagonally) receives no triangle and so no stiffness contribution at all – an all-zero matrix row, which is singular. `assembleFEM` detects any DOF untouched by every triangle and pins it to $u = 0$ exactly as a boundary DOF (Section 7), which is also the geometrically honest choice: such a node is not meaningfully inside the FEM mesh's coverage of Ω.

7 Boundary conditions: symmetric Dirichlet elimination

Both assemblers hand their (row, column, value) triplets and load vector to the same routine, `applyDirichletZero`, *before* the triplets are compressed into a CSR matrix. For every boundary DOF d:

1. every triplet with row d or column d is dropped;
2. a single triplet $(d, d, 1)$ is inserted;
3. $b_d \leftarrow 0$.

Because the boundary value is homogeneous ($g \equiv 0$), this is the full story: the general technique for an inhomogeneous Dirichlet value $g_d \neq 0$ also moves a correction term $b_j \mathrel{-}= A_{jd}\, g_d$ into every other row j before dropping column d; here that term is identically zero, so dropping the column *is* the complete correction and no extra pass is needed.

Why this keeps the system SPD. The un-eliminated FDM and FEM matrices are each symmetric by construction (Sections 5, 6) and positive semi-definite – positive definite on H_0^1, which is exactly the subspace left over once the boundary DOFs are removed. Step 1 above deletes the row *and* column of every boundary DOF, which is precisely restricting the bilinear form to that subspace; symmetry survives because rows and columns are removed together. Step 2 then appends an identity block on the deleted DOFs. The resulting matrix is block-diagonal,

$$A = \begin{pmatrix} A_{\text{int}} & 0 \\ 0 & I \end{pmatrix}, \tag{11}$$

with A_{int} symmetric positive definite (it is the Galerkin/finite-difference operator restricted to H_0^1) and I trivially SPD. A block-diagonal matrix of SPD blocks is SPD. This is the fact that licenses using **the same Conjugate Gradient solver, unmodified, for both discretizations** – CG's convergence theory (Section 8.2) requires exactly this property, and nothing in the derivation above is specific to either FDM or FEM.

8 Linear algebra

8.1 CSR storage

`CSRMatrix::fromTriplets` takes an unordered list of $(row, col, value)$ triplets – produced directly by assembly, with no effort made to avoid emitting the same (row, col) pair more than once – and:

1. sorts the triplets lexicographically by (row, col);
2. sums the values of any triplets sharing a (row, col) pair (this is what lets multiple elements or stencil contributions targeting the same matrix entry simply be appended to the triplet list rather than requiring assembly to look up and update an existing entry);

3. lays out the result as three flat arrays: `rowPtr` (length $n+1$), `colIdx`, `values` (both length nnz) – the standard CSR format.

A matrix-vector product then costs $O(\text{nnz})$:

$$y_r = \sum_{k=\texttt{rowPtr}[r]}^{\texttt{rowPtr}[r+1]-1} \texttt{values}[k] \cdot x_{\texttt{colIdx}[k]}. \tag{12}$$

`matvec` is the only operation the iterative solver needs from the matrix – CG never forms, factors, or otherwise touches A except through this product.

8.2 Conjugate Gradient

CG solves $A\mathbf{x} = \mathbf{b}$ for SPD A by minimizing the energy norm $\|\mathbf{x} - \mathbf{x}^\star\|_A = \sqrt{(\mathbf{x} - \mathbf{x}^\star)^\top A(\mathbf{x} - \mathbf{x}^\star)}$ over successively larger Krylov subspaces $\mathcal{K}_k = \text{span}\{\mathbf{r}_0, A\mathbf{r}_0, \ldots, A^{k-1}\mathbf{r}_0\}$, $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$. Starting from $\mathbf{x}_0 = \mathbf{0}$ (the natural guess here, since the true solution is 0 on every boundary DOF and $u_0 = 0$ is a reasonable prior on interior DOFs too), each iteration:

1. computes $\mathbf{Ap}$ (one `matvec`);
2. takes the energy-optimal step $\alpha = \dfrac{\mathbf{r}^\top \mathbf{z}}{\mathbf{p}^\top A\mathbf{p}}$ along the current search direction $\mathbf{p}$, where $\mathbf{z} = M^{-1}\mathbf{r}$ is the preconditioned residual;
3. updates $\mathbf{x} \mathrel{+}= \alpha\mathbf{p}$, $\mathbf{r} \mathrel{-}= \alpha A\mathbf{p}$;
4. forms the next search direction $\mathbf{p} \leftarrow \mathbf{z} + \beta\mathbf{p}$, $\beta = \dfrac{\mathbf{r}_{\text{new}}^\top \mathbf{z}_{\text{new}}}{\mathbf{r}^\top \mathbf{z}}$, A-conjugate to every previous direction.

`ConjugateGradient.hpp` uses the **Jacobi preconditioner** $M = \text{diag}(A)$ – the cheapest preconditioner available, needing only the diagonal already exposed by `CSRMatrix::diagonal()`, and effective here because the Poisson matrices built in Sections 5–6 are diagonally dominant (FDM exactly, FEM for reasonably shaped triangles), so $M^{-1}A$ is noticeably better conditioned than A itself.

Convergence bound. With exact arithmetic, CG reaches the exact solution within n steps (n = number of DOFs), since the Krylov subspaces can grow no larger. In practice it is used as a genuinely iterative method because the energy-norm error after k iterations satisfies

$$\|\mathbf{x}_k - \mathbf{x}^\star\|_A \;\le\; 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|_A, \qquad \kappa = \kappa(M^{-1}A) = \frac{\lambda_{\max}}{\lambda_{\min}}, \tag{13}$$

the condition number of the (preconditioned) matrix. The unpreconditioned 5-point/P1 Poisson operator has $\kappa = O(h^{-2}) = O(N^2)$, so (13) predicts iteration counts growing like $O(N)$ for a fixed target accuracy – consistent with the iteration counts `main.cpp` reports growing visibly with grid resolution in the runs described in Section 9. The solver tracks exactly the quantity (13) controls, the relative residual $\|\mathbf{b} - A\mathbf{x}_k\|_2 / \|\mathbf{b}\|_2$, and stops once it falls below the convergence tolerance (default 10^{-10}) – tight enough that solver error is negligible next to the discretization error being compared in Section 9.

9 Why the comparison is a correctness check

The two discretizations are derived by genuinely different arguments – FDM from a Taylor-series local truncation at each grid point, FEM from a global energy/weak-form minimization over a triangulated mesh – sharing nothing except:

- the same continuous target, equation (1);
- the same `Grid2d` instance, hence the same DOF indexing (Section 4.2);
- the same symbolic source f (Section 1);
- the same homogeneous-Dirichlet elimination routine (Section 7);
- the same CSR + CG linear solve (Section 8).

Both are consistent, convergent approximations of the same elliptic problem: as $h \to 0$, $u_h^{\mathrm{FDM}} \to u$ and $u_h^{\mathrm{FEM}} \to u$ in the appropriate norms, each with its own $O(h^2)$ (energy-norm $O(h)$, L^2-norm typically $O(h^2)$ for FEM) discretization error. Two independent, convergent approximations of the same limit must therefore agree with each other to within the sum of their individual discretization errors – i.e. $\|u_h^{\mathrm{FDM}} - u_h^{\mathrm{FEM}}\|$ should itself shrink as $h \to 0$, even though neither solution is exactly u. **This is the actual claim being tested**, not merely "two numbers happen to be close." A bug in either assembler – a sign error, a missing neighbor, a wrong quadrature weight – would generically *not* shrink with h (it would show up as an $O(1)$ or wrong-order discrepancy between the two fields), which is exactly what the max-pointwise-difference figure `main.cpp` prints is designed to catch.

The runs recorded when the module was built (circle/`sine`, N=64; L-shape/`bump`, N=48; plus/`constant`, N=64; square/`checker`, N=80) all show maximum pointwise disagreement on the order of 10^{-5}, two to three orders of magnitude below the solution magnitudes themselves, concentrated away from the boundary where both methods are at their most accurate – the expected signature of two correct, convergent, independently-derived discretizations of the same problem rather than of a coincidence.

10 The interactive desktop window

`src/compare/gui` wraps the same assembly and solve functions used by `main.cpp` in a standalone FLTK window (`compare_gui`), so the comparison can be explored live instead of by re-running the CLI and re-opening a PNG. It is a thin, purely presentational layer: it adds no new numerics beyond the two things specified in this section – when to (re-)solve, and how to project a scalar field onto the screen.

10.1 Controls and the solve-on-demand model

`CompareWindow` exposes five controls in a sidebar: **Domain**, **Source function** f, **Method** (FDM only / FEM only / both), **Mesh resolution** N, and **View** (Heatmap / 3D surface). Changing a control does not, by itself, trigger any computation – `CompareWindow::solve()` runs only when the **Solve** button is pressed (or once, automatically, when the window first opens), and it is exactly the sequence already specified: build a fresh `Grid2d` for the chosen domain and N (Section 4.2), call `assembleFDM` and/or `assembleFEM` (Sections 5–6) with the chosen f, solve each with `conjugateGradient` (Section 8.2), and hand the resulting node-value vector(s) to `HeatmapView` for drawing. The **View** control is the one exception: it only changes how the most recently solved field is drawn, so its callback redraws immediately without re-solving anything.

10.2 Rendering: two views of the same vector

`HeatmapView` draws whatever `Panel`s (a title, a `Grid2d`, and a node-value vector) it was last given, in one of two modes, both reusing `pde::viridis` (`src/gui/include/ColorMap.hpp`, the same colormap `pde_studio` already uses) to map a normalized field value $t = (v - v_{\min})/(v_{\max} - v_{\min}) \in [0, 1]$ to a color.

Heatmap mode is the direct 2D analogue of `plot_compare.py`'s `pcolormesh` panels: each active grid cell is filled with its color, inactive cells are left as background.

3D surface mode draws the same field as a shaded surface using a fixed, non-interactive *axonometric* (oblique/isometric-style) projection – a classical, simple technique for turning a 3D shape into a 2D drawing with no perspective divide and no camera matrix, deliberately avoiding an OpenGL/`Fl_Gl_Window` dependency anywhere else in this module. A world point (x, y, z), with domain coordinates $x, y \in [0, 1]$ and a per-panel normalized height $z = t \in [0, 1]$, projects to the 2D point

$$\begin{pmatrix} s_x \\ s_y \end{pmatrix} = \begin{pmatrix} (x - y)\cos\alpha \\ (x + y)\sin\alpha \; - \; \lambda z \end{pmatrix}, \qquad \alpha = 30^\circ, \quad \lambda = 0.6, \tag{14}$$

implemented verbatim as `project()` in `HeatmapView.cpp`. The $(x - y)$ and $(x + y)$ combination is the standard 30° isometric rotation of the xy-plane (it sends the unit square to a diamond, matching the familiar "cube corner" look of isometric drawings); z is subtracted directly, undistorted by the rotation, since in this convention height is already the screen's vertical axis. λ is a fixed *vertical exaggeration* constant, not a physical unit – z is already normalized to $[0, 1]$ per panel (Section 5's solutions range over values as small as 10^{-2}), so λ simply sets how tall the rendered bump looks relative to the 1×1 footprint, independent of the solution's actual magnitude.

Equation (14) is evaluated at the 8 corners of the unit cube $\{0, 1\} \times \{0, 1\} \times \{0, 1\}$ to get the projection's bounding box, which is then linearly fit to the panel's pixel rectangle (the same fit-then-map idea as the heatmap's cell sizing, just over the projected bounding box instead of grid indices directly). Each fully-active grid cell (all four corners active, the same condition `assembleFEM` uses for its two-triangle case; a partially-active cell is skipped entirely in this mode) becomes one filled, outlined quadrilateral with its four corners individually elevated by their own node's height – so a cell is not flat unless its four corner values happen to agree, exactly like a bilinear surface patch – colored by the average of its four corner heights.

Depth ordering. With no z-buffer, cells must be drawn back-to-front by hand (the *painter's algorithm*). Under projection (14), a cell's screen depth (ignoring the small z term) increases monotonically with $i + j$: moving to a larger grid index along either axis moves *away* from the $(0, 0)$ corner and *down* the screen ($\partial s_y/\partial x > 0$ and $\partial s_y/\partial y > 0$ both hold by inspection of (14), since the $-\lambda z$ term is $O(\lambda) = O(1)$ while the depth coordinate itself ranges over $[0, 2\sin\alpha] = [0, 1]$, so it dominates whenever the field does not vary by more than order 1 between adjacent cells). `drawPanelSurface3D` therefore draws cells in ascending order of $i + j$, which is exact for a flat or monotone surface and a reasonable approximation for the smooth, single-featured fields in `SourceFunctions.hpp` – it is not a substitute for a real depth buffer on an arbitrarily jagged field, which this module's sources do not produce.

11 Known approximations and limitations

- **Staircase boundary.** The circle's true boundary is curved; both discretizations approximate it with the grid-aligned boundary of Section 4.2, which is only $O(h)$ accurate at the boundary itself (the axis-aligned domains – square, L-shape, plus – have no such error, since their true boundaries already lie on grid lines for suitable N). A higher-order treatment (e.g. Shortley–Weller for FDM, curved/cut elements for FEM) is out of scope here; see `docs/ROADMAP.md` Phase 9 for where that belongs in the broader project.
- **Load-vector quadrature.** The one-point centroid rule (10) is exact for constant f and only approximate otherwise; for highly oscillatory f relative to the mesh (e.g. `checker` at low N)

this adds an error term beyond the usual P1 interpolation error.

- **Fixed source geometry.** f is restricted to the five expressions in Section 3; any `pde::Expr` can be substituted by editing `SourceFunctions.hpp`, with no other code changes required.
- **Scope boundary with the rest of the project.** This module does not use `Domain2d`, `mesh2d`, or the existing `src/gui` app (`AppWindow`/`IPDEProblem`), and does not depend on Eigen; its own interactive window (Section 10) is a separate FLTK target (`compare_gui`), built from the same solver code but sharing no window-management code with `pde_studio` beyond the reused `ColorMap.hpp`.
- **3D view is a fixed projection, not a camera.** Equation (14) is a single, non-interactive viewpoint – there is no rotation, pan, or zoom, unlike a true OpenGL-based 3D view. The painter's-algorithm depth ordering (Section 10) is also only exact for the smooth, single-featured source functions this module ships with; a sufficiently oscillatory f (e.g. `checker` at low N) could in principle produce a cell whose true depth ordering disagrees with the $i + j$ approximation.